

S E M E S T E R — I

Introduction to Computer Science and Python Programming

Complete Unit-Wise Study Notes
(Theory + Concepts + Diagrams + Exam Preparation Guide)

Prepared by	Manthan Sahu
Program	Computer Science Student
University	Rajendra University
Course	Introduction to Computer Science and Python Programming
Covers	Unit I – Unit IV, I-Lab Programs, Exam Guide

A Self-Prepared Reference for Learning, Revision & Examination

How to Use These Notes

UNIT I – Introduction to Computer Science

1. Introduction to Computers
2. Computer System Hardware
 - 2.1 CPU (Central Processing Unit)
 - 2.2 Memory Unit
 - 2.3 Storage Devices
3. Computer Memory
 - 3.1 Memory Hierarchy
 - 3.2 CPU Registers
 - 3.3 Cache Memory
 - 3.4 Primary Memory
 - 3.5 Secondary Memory
 - 3.6 Access Methods
4. Input & Output Units
5. Computer Software
 - Types of Software
6. Computer Network
 - 6.1 Network Types
 - 6.2 Internet and Its Applications
 - 6.3 Network Devices

★ Unit I Quick Revision

🔍 Unit I Important Questions

- ◆ 1 Mark Questions
- ◆ 2 Mark Questions
- ◆ 5 Mark Questions
- ◆ 8 Mark Questions

UNIT II – Program Development and Programming Languages

1. Program Development
2. Algorithm
3. Flowchart
4. Programming Paradigms & Design
 - 4.1 Structured Programming
 - 4.2 Object-Oriented Programming (OOP)
 - 4.3 Top-Down and Bottom-Up Design Approaches
 - 4.4 Characteristics of a Good Program
5. Programming Languages
 - 5.1 Low-Level vs High-Level Languages
 - 5.2 Generations of Programming Languages
6. Translators: Assembler, Compiler, Interpreter
 - Linker, Loader, Editors, IDE
7. Programming Errors

★ Unit II Quick Revision

🔍 Unit II Important Questions

- ◆ 1 Mark Questions
- ◆ 2 Mark Questions
- ◆ 5 Mark Questions
- ◆ 8 Mark Questions

UNIT III – Python Basics, Strings, Console I/O, and Control Instructions

1. Introduction to Python & Getting Started
2. Python Basics
 - 2.1 Identifiers
 - 2.2 Keywords
 - 2.3 Python Types / Basic Types
 - 2.4 Mutable and Immutable Types
 - 2.5 Integer & Float Ranges
 - 2.6 Variable Type & Assignment
 - 2.7 Arithmetic Operators
 - 2.8 Precedence & Associativity
 - 2.9 Conversions (Type Casting)
 - 2.10 Built-in Functions
 - 2.11 Modules
 - 2.12 Container Types
 - 2.13 Comments & Indentation
 - 2.14 Multi-lining
 - 2.15 Classes & Objects, Multiple Objects
3. Strings
 - 3.1 Accessing String Elements (Indexing & Slicing)
 - 3.2 Properties
 - 3.3 Built-in Functions & Methods
 - 3.4 Conversions

3.5 Comparisons	
4. Console I/O	
4.1 I/O Operations	
4.2 Formatted Printing	
5. Decision Control Instruction	
5.1 Logical Operators	
5.2 Conditional Expressions (if / elif / else)	
5.3 all() and any()	
5.4 Receiving Input (in the context of decisions)	
5.5 pass Statement	
6. Repetition Control Instruction (Loops)	
6.1 Types of Loops	
6.2 break and continue	
6.3 else Block of a Loop	
★ Unit III Quick Revision	
🔍 Unit III Important Questions	
◆ 1 Mark Questions	
◆ 2 Mark Questions	
◆ 5 Mark Questions	
◆ 8 Mark Questions	
UNIT IV – Data Structures, Comprehensions, and Functions	
1. Lists	
2. Sets	
3. Tuples	
4. Dictionaries	
5. Comprehensions (List, Set, Dictionary)	
6. Functions	
6.1 Built-in vs User-Defined Functions	
6.2 Invoking Functions & Unpacking Arguments	
6.3 Recursive Function	
6.4 Iteration vs Recursion	
★ Unit IV Quick Revision	
🔍 Unit IV Important Questions	
◆ 1 Mark Questions	
◆ 2 Mark Questions	
◆ 5 Mark Questions	
◆ 8 Mark Questions	
LAB PROGRAMS – Programming Using Python (I-Lab)	
📖 EXAM PREPARATION GUIDE (Full Syllabus)	
1. Most Important Topics from the Entire Syllabus	
2. Most Important Definitions	
3. Most Important Diagrams	
4. Most Important Formulas / Syntax	
5. Most Important Differences (Tables to Revise)	
6. Most Repeated/Expected Question Types	
7. Unit-Wise Important Questions (Summary List)	
8. Last-Minute Revision Notes	
9. Exam Strategy	
10. Top Questions to Prepare First (★ Very Important, across all units)	

How to Use These Notes

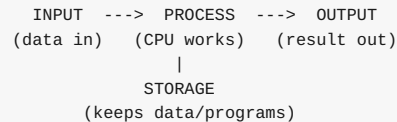
These notes cover **every topic and sub-topic** listed in the syllabus for *Introduction to Computer Science and Python Programming*, unit by unit. Each topic is explained from the basics, with definitions, working, examples, real-life analogies, diagrams (drawn using simple ASCII/box diagrams), key points, common mistakes, comparisons, and exam-oriented notes. Every unit ends with a **Quick Revision** page and **Important Questions** (1, 2, 5, 8 marks) marked by priority (★ Very Important, 📖 Important, ⚡ Must Know). The Lab Programs (24 programs) and a final **Exam Preparation Guide** are included at the end.

UNIT I – Introduction to Computer Science

1. Introduction to Computers

Simple Definition: A computer is an electronic machine that takes data (input), processes it according to instructions, and gives back useful information (output).

Concept Explanation (from basics): Think of a computer as a very fast, very obedient assistant. It cannot think on its own — it only follows instructions (called a *program*) step by step. The basic cycle every computer follows is:



Why/Where It Is Used: Computers are used everywhere — banking, education, medicine, communication, entertainment, research, and industry — because they process data accurately and at very high speed.

Generations of Computers

Generation	Period	Technology Used	Example
1st	1940–1956	Vacuum Tubes	ENIAC, UNIVAC
2nd	1956–1963	Transistors	IBM 1401
3rd	1964–1971	Integrated Circuits (IC)	IBM 360
4th	1971–present	Microprocessors (VLSI)	Personal Computers
5th	Present–future	Artificial Intelligence	AI-based systems

Types of Computers

Type	Description	Example
Supercomputer	Extremely fast, used for complex scientific calculations	Weather forecasting systems
Mainframe	Large computers used by big organizations for bulk data processing	Bank servers
Minicomputer	Mid-sized, less powerful than mainframe	Small business servers
Microcomputer (PC)	Personal, single-user computer	Desktop, Laptop
Embedded Computer	Built into another device to perform a specific task	Washing machine controller

Real-Life Analogy: A computer is like a chef in a kitchen — raw vegetables (input) are processed (cooking = processing) using kitchen equipment (CPU) to produce a finished dish (output), and leftover ingredients are kept in the fridge (storage) for later use.

Key Takeaways - Computer = Input + Process + Output + Storage. - It works only on the instructions given to it (it has no thinking of its own). - Five generations exist, each defined by the core electronic technology used.

Common Mistakes - Confusing “generation” (technology era) with “type” (size/purpose) of computer. - Thinking a computer can “think” — it only executes given instructions (no independent intelligence, except in AI-assisted systems which still follow programmed logic).

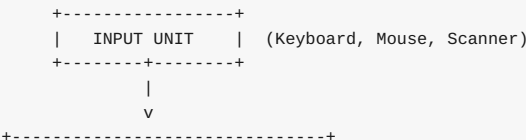
Exam-Oriented Notes: Remember the generation table (technology + example) — a favourite 2-mark/5-mark question. Also remember the 5 types of computers with one example each.

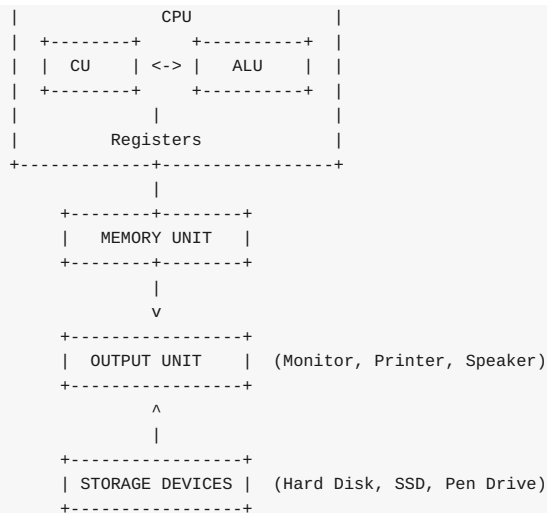
Important Terms: Data, Information, Program, Hardware, Software, IC (Integrated Circuit), VLSI (Very Large Scale Integration).

2. Computer System Hardware

Simple Definition: Hardware refers to all the physical, touchable parts of a computer.

Block Diagram of a Computer





2.1 CPU (Central Processing Unit)

Definition: The CPU is the “brain” of the computer — it fetches, decodes, and executes instructions.

Parts of CPU: 1. **Control Unit (CU):** Directs and coordinates all activities of the computer; tells other units what to do and when (does not process data itself). 2. **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (add, subtract, multiply, divide) and logical operations (comparisons like <, >, =). 3. **Registers:** Very small, very fast temporary storage locations inside the CPU used to hold data currently being processed.

Working (Fetch-Decode-Execute Cycle):

```

FETCH instruction from memory
-> DECODE it (understand what to do)
-> EXECUTE it (perform the operation)
-> repeat for next instruction
  
```

Real-Life Analogy: The CPU is like the brain of a student solving a maths problem — the Control Unit is like the part of the brain deciding “what to do next,” and the ALU is like the part actually doing the calculation.

2.2 Memory Unit

Covered in detail in Topic 3 (Computer Memory) below, since the syllabus lists memory separately with more depth.

2.3 Storage Devices

Definition: Devices used to permanently store data even when the computer is powered off (non-volatile).

Functions: Save programs and data permanently; allow retrieval later; provide large capacity at low cost compared to primary memory.

Examples: Hard Disk Drive (HDD), Solid State Drive (SSD), USB Pen Drive, CD/DVD, Memory Card.

Key Takeaways - Hardware = physical parts (can be touched). - CPU = CU + ALU + Registers. - Storage devices are non-volatile (data stays even without power); Primary memory (RAM) is mostly volatile.

Common Mistakes: Students often confuse “Memory Unit” (fast, temporary, e.g. RAM) with “Storage Devices” (slower, permanent, e.g. Hard Disk).

Exam-Oriented Notes: Draw the block diagram of a computer — very commonly asked for 5/8 marks. Also be ready to explain CU vs ALU as a 2-mark difference question.

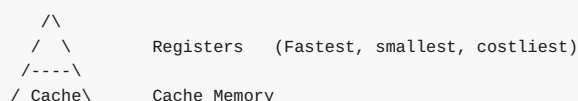
Important Terms: CPU, CU, ALU, Registers, Fetch-Decode-Execute cycle, Non-volatile, Volatile.

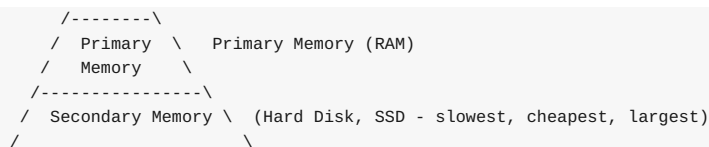
3. Computer Memory

Simple Definition: Memory is the part of the computer that stores data and instructions, either temporarily or permanently.

3.1 Memory Hierarchy

Concept: Memory is arranged in a pyramid — from fastest & most expensive (top, small size) to slowest & cheapest (bottom, large size).





3.2 CPU Registers

Small, ultra-fast storage locations built directly into the CPU. Used to hold data/instructions currently being executed. Examples: Accumulator, Program Counter, Instruction Register.

3.3 Cache Memory

A small, very fast memory placed between the CPU and RAM. It stores frequently used data so the CPU doesn't have to wait for slower RAM every time. **Analogy:** Cache is like keeping your most-used textbooks on your study table (fast access) instead of walking to the library (RAM) every time.

3.4 Primary Memory

- **RAM (Random Access Memory):** Volatile (data lost when power is off). Temporarily holds data/programs currently in use.
- **ROM (Read Only Memory):** Non-volatile. Stores permanent instructions (like startup instructions – BIOS).

3.5 Secondary Memory

Non-volatile, large-capacity storage used for permanent data storage (Hard Disk, SSD, CD/DVD, Pen Drive).

3.6 Access Methods

Access Method	Description	Example
Sequential Access	Data accessed in order, one after another from the start	Magnetic Tape
Direct/Random Access	Any data can be accessed directly without going through others	RAM, Hard Disk
Indexed Sequential Access	Combination — uses an index to jump near the data, then sequential search	Indexed files on disk

Comparison Table: RAM vs ROM

Feature	RAM	ROM
Full form	Random Access Memory	Read Only Memory
Volatility	Volatile (data lost on power off)	Non-volatile
Read/Write	Read and Write	Mostly Read only
Use	Temporary working memory	Stores permanent boot instructions

Key Takeaways - Speed order (fastest → slowest): Registers > Cache > RAM > Secondary Memory. - Size order (smallest → largest): Registers < Cache < RAM < Secondary Memory. - RAM is volatile; ROM and Secondary Memory are non-volatile.

Common Mistakes: Thinking Cache and RAM are the same thing; thinking ROM can be freely rewritten like RAM.

Exam-Oriented Notes: The Memory Hierarchy diagram + RAM vs ROM difference table are extremely common exam questions.

Important Terms: Volatile, Non-volatile, RAM, ROM, Cache, Registers, Access Methods.

4. Input & Output Units

Simple Definition: Input devices are used to enter data into the computer; output devices are used to display or produce the processed result.

Input Devices	Function
Keyboard	Enter text/characters
Mouse	Point, click, select on screen
Scanner	Convert paper documents/images into digital form
Microphone	Enter sound/voice
Joystick	Used mainly for gaming, controls movement

Output Devices	Function
Monitor (VDU)	Displays visual output on screen
Printer	Produces output on paper (hard copy)
Speaker	Produces sound output
Plotter	Prints large graphics/drawings

Real-Life Analogy: Input devices are like your mouth speaking to someone (giving information), and output devices are like your ears listening to the reply (receiving the result).

Key Takeaways: Input → data goes IN; Output → result comes OUT. Some devices can do both (e.g., Touch Screen).

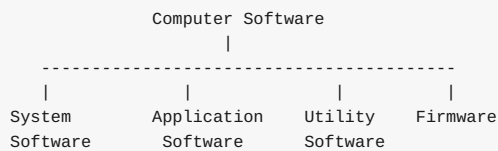
Exam-Oriented Notes: Be ready to classify a list of devices as input/output — common 1-mark/2-mark question.

Important Terms: VDU (Visual Display Unit), Hard copy, Soft copy.

5. Computer Software

Simple Definition: Software is the set of programs/instructions that tell the hardware what to do. Unlike hardware, it cannot be touched.

Types of Software



1. **System Software:** Manages and controls the hardware so application software can run. Example: Operating System (Windows, Linux), Device Drivers.
2. **Application Software:** Designed for the end-user to perform specific tasks. Example: MS Word, browsers, games.
3. **Utility Software:** Helps maintain and optimize the computer's performance. Example: Antivirus, Disk Cleanup, File compression tools.
4. **Firmware:** Permanent software programmed into hardware (usually stored in ROM) that controls low-level device operations. Example: BIOS.

Comparison: System vs Application Software

Feature	System Software	Application Software
Purpose	Manages hardware/system resources	Performs specific user tasks
Example	Operating System	MS Excel, Photoshop
Dependency	Runs independently of applications	Depends on system software to run

Real-Life Analogy: System software is like the manager of a building (keeps everything running), while application software is like a tenant using a specific room for a specific purpose (e.g., a bakery).

Key Takeaways: Software = intangible; 4 types = System, Application, Utility, Firmware.

Common Mistakes: Calling Utility software “Application software” — utility software specifically maintains/optimizes the system, not perform user tasks like word processing.

Exam-Oriented Notes: System vs Application software difference is a classic 2-mark/5-mark question.

Important Terms: OS, Device Driver, BIOS, Antivirus.

6. Computer Network

Simple Definition: A computer network is a group of two or more computers connected together to share data and resources.

Importance of Networking: Enables resource sharing (printers, files), fast communication (email, chat), centralized data management, and remote access.

6.1 Network Types

Type	Full Form	Coverage Area	Example
LAN	Local Area Network	Small area (one building/campus)	Office network
MAN	Metropolitan Area Network	A city	Cable TV network

WAN	Wide Area Network	Large area (country/world)	Internet
-----	-------------------	----------------------------	----------

Visualization:

LAN (one office) < MAN (one city) < WAN (whole world)
small medium largest

6.2 Internet and Its Applications

The Internet is a global WAN connecting millions of networks worldwide. Applications: Email, World Wide Web (WWW), Online banking, E-commerce, Social media, Video conferencing, Cloud storage.

6.3 Network Devices

Device	Function
Repeater	Regenerates/amplifies a weak signal so it can travel further
Bridge	Connects two LAN segments and filters traffic between them
Hub	Simply broadcasts incoming data to all connected devices (no intelligence)
Switch	Sends data only to the specific device it is meant for (smarter than hub)
Router	Connects different networks and directs data between them (e.g., LAN to Internet)
Gateway	Connects networks using different protocols, translating between them

Real-Life Analogy: A Hub is like shouting a message to an entire room (everyone hears it); a Switch is like handing a note directly to the one person it is addressed to; a Router is like a postal service directing mail between different cities (networks).

Key Takeaways: LAN < MAN < WAN in coverage area. Hub = broadcasts to all; Switch = sends to specific device; Router = connects different networks.

Common Mistakes: Confusing Hub and Switch (Hub has no intelligence, Switch does); confusing Router (connects networks) with Switch (connects devices within one network).

Exam-Oriented Notes: Table of network devices with one-line function each is a very common 5-mark question. LAN/MAN/WAN comparison is a common 2-mark question.

Important Terms: LAN, MAN, WAN, Repeater, Bridge, Hub, Switch, Router, Gateway, WWW.

☆ Unit I Quick Revision

- Computer = Input + Process + Output + Storage; 5 generations (Vacuum Tubes → Transistors → IC → Microprocessor → AI).
- 5 types of computers: Super, Mainframe, Mini, Micro, Embedded.
- CPU = Control Unit (CU) + Arithmetic Logic Unit (ALU) + Registers.
- Memory Hierarchy (fast → slow, small → large): Registers → Cache → Primary Memory (RAM/ROM) → Secondary Memory.
- RAM = volatile, temporary; ROM = non-volatile, permanent.
- Access methods: Sequential, Direct/Random, Indexed Sequential.
- Input devices bring data in; Output devices give results out.
- Software types: System, Application, Utility, Firmware.
- Network types: LAN (small) < MAN (city) < WAN (world, e.g. Internet).
- Network devices: Repeater (amplify), Bridge (connect LAN segments), Hub (broadcast), Switch (targeted send), Router (connect networks), Gateway (translate protocols).

? Unit I Important Questions

◆ 1 Mark Questions

- ✎ Define a computer.
- ✎ Expand: CPU, ALU, CU, RAM, ROM, LAN, MAN, WAN.
- ✎ Name the five generations of computers.
- ✎ What is firmware?
- 🗑 What is the difference between hardware and software?
- ✎ Name any two input and two output devices.

◆ 2 Mark Questions

- 🗑 Differentiate between RAM and ROM.
- 🗑 Differentiate between System Software and Application Software.
- ★ Differentiate between a Hub and a Switch.

- 🖨️ What is Cache memory? Why is it used?
- ✨ List the types of computers with one example each.
- 🖨️ What is the difference between LAN and WAN?

🔹 5 Mark Questions

- ✨ Draw and explain the block diagram of a computer.
- ✨ Explain the memory hierarchy with a diagram.
- 🖨️ Explain any five network devices with their functions.
- 🖨️ Explain the types of software with examples.
- 🖨️ Explain the generations of computers in a table.

🔹 8 Mark Questions

- ✨ Explain in detail the components of CPU and the fetch-decode-execute cycle with a diagram.
- ✨ Explain computer memory: hierarchy, types (Primary/Secondary), and access methods in detail.
- 🖨️ Explain computer networks: importance, types (LAN/MAN/WAN), and network devices with diagrams.
- 🖨️ Explain the different types of computer software with real examples of each.

UNIT II – Program Development and Programming Languages

1. Program Development

Simple Definition: Program development is the process of converting a real-world problem into a working computer program through systematic steps.

Steps in Program Development:

Problem Analysis -> Program Design -> Coding -> Testing -> Debugging -> Documentation -> Maintenance

1. **Problem Analysis:** Understanding what the problem is, what inputs are given, and what output is expected.
2. **Program Design:** Planning the logic/steps to solve the problem (using algorithm/flowchart) before writing actual code.
3. **Development:** Writing the actual code in a programming language, then testing and debugging it.

Real-Life Analogy: Building software is like constructing a house — first you analyse the family’s needs (problem analysis), then draw a blueprint (design), then build it (coding), then inspect it for faults (testing/debugging).

Key Takeaways: Never jump directly to coding — always analyse and design first. This saves time and avoids major errors later.

Exam-Oriented Notes: Listing the steps of program development in order is a common 2-mark question.

2. Algorithm

Simple Definition: An algorithm is a finite set of well-defined, step-by-step instructions to solve a problem.

Concept Explanation: Before writing code in any language, we first think out the logic in plain steps — this is the algorithm. It is language-independent, meaning the same algorithm can be implemented in Python, C, Java, etc.

Characteristics of a good algorithm: Finite (must end), Definite (each step is clear, unambiguous), has Input, has Output, and is Effective (each step must be simple enough to be carried out).

Pseudocode: A way of writing an algorithm using a mix of English and programming-like structure (not tied to any real programming language syntax).

Easy Example — Algorithm to find the sum of two numbers:

```
Step 1: Start
Step 2: Read two numbers A and B
Step 3: Compute SUM = A + B
Step 4: Print SUM
Step 5: Stop
```

Pseudocode Example:

```
BEGIN
  READ a, b
  sum <- a + b
  PRINT sum
END
```

Real-Life Analogy: An algorithm is like a cooking recipe — a numbered list of steps that, if followed exactly, always produces the same dish (result).

Control Structures (used while writing algorithms/programs):

Structure	Meaning	Example
Sequence	Steps executed one after another in order	Step1, Step2, Step3
Selection (Decision)	Choosing a path based on a condition	if...else
Iteration (Loop)	Repeating steps until a condition is met	for, while

Key Takeaways: Algorithm = logic in plain steps; must be finite and unambiguous.

Common Mistakes: Making an algorithm too vague (e.g. “process the data” is not a proper step — steps must be precise).

Exam-Oriented Notes: Write algorithm + pseudocode for simple problems (sum, largest of 3 numbers, factorial) — very common 5-mark question.

Important Terms: Pseudocode, Sequence, Selection, Iteration.

3. Flowchart

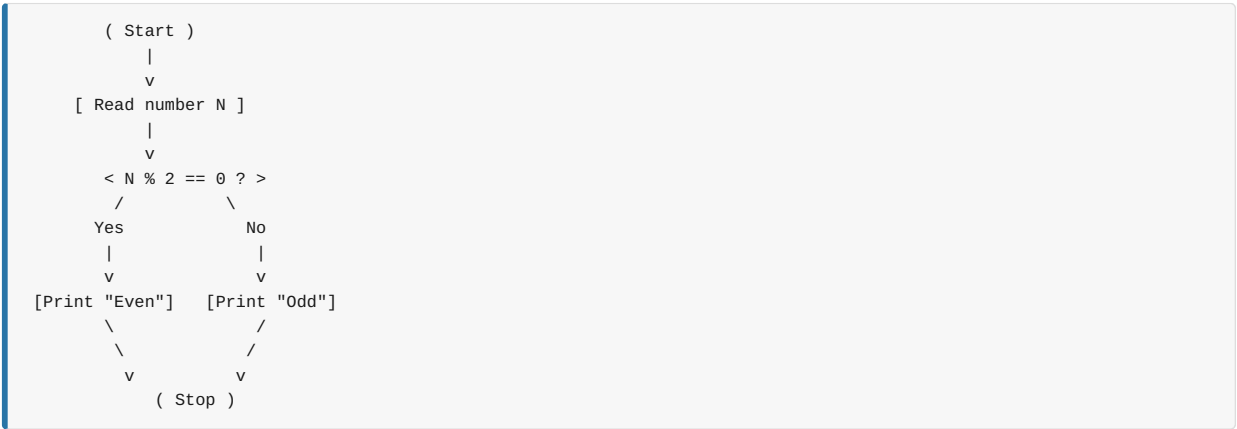
Simple Definition: A flowchart is the pictorial (graphical) representation of an algorithm using standard symbols.

Why Used: Easier to visualize the logic and flow of control at a glance compared to reading plain text steps.

Flowchart Symbols

Symbol Name	Shape	Purpose
Terminal (Start/Stop)	Oval	Marks beginning and end of flowchart
Input/Output	Parallelogram	Represents input or output operation
Process	Rectangle	Represents a processing step/calculation
Decision	Diamond	Represents a condition/decision point (yes/no)
Flow lines	Arrow	Shows the direction of flow/control
Connector	Circle	Connects different parts of a flowchart (especially across pages)

Example — Flowchart to check if a number is even or odd:



Real-Life Analogy: A flowchart is like a road map with junctions (decision diamonds) showing which direction to take based on conditions.

Key Takeaways: Always start/end with an oval (Terminal); diamonds are only for decisions; arrows always show the direction of flow.

Common Mistakes: Using a rectangle for a decision instead of a diamond; forgetting arrows between symbols.

Exam-Oriented Notes: Drawing simple flowcharts (largest of 2/3 numbers, even-odd check, sum of N numbers) is a very common 5-mark question.

Important Terms: Terminal, Process box, Decision box, Connector, Flow lines.

4. Programming Paradigms & Design

Simple Definition: A programming paradigm is a style/approach of writing and organizing computer programs.

4.1 Structured Programming

Divides a program into small, manageable blocks/modules (functions), each performing one task, executed using sequence, selection, and iteration only (no random jumps like `goto`). **Example use:** C language programs organized into functions.

4.2 Object-Oriented Programming (OOP)

Organizes a program around **objects** (which bundle data and behaviour together) rather than just functions. Real-world entities are modelled as objects belonging to classes. **Example use:** Python, Java, C++.

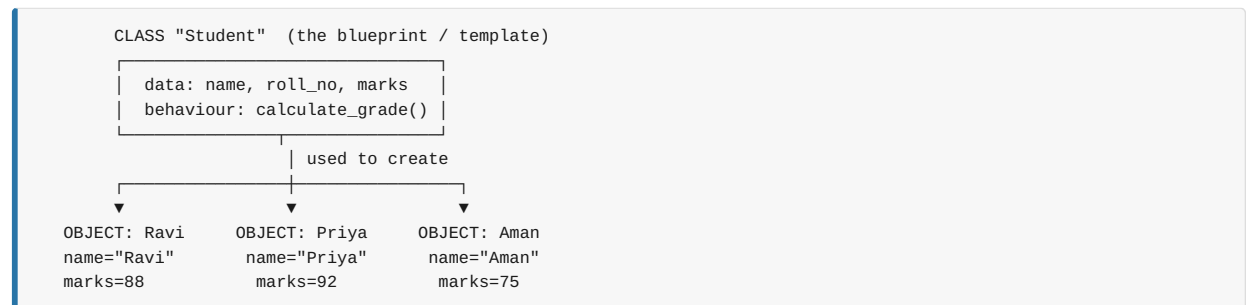
Comparison Table

Feature	Structured Programming	Object-Oriented Programming
Focus	Functions/procedures	Objects (data + behaviour)
Data Security	Data is less protected (often global)	Data is encapsulated (protected) inside objects
Reusability	Limited (via functions)	High (via classes, inheritance)

Example Languages	C, Pascal	Python, Java, C++
-------------------	-----------	-------------------

Deeper Concept — the intuition behind OOP: In **structured programming**, data and the functions that operate on it are kept **separate**. For example, a student’s name/marks might sit in one set of variables, while a totally separate function calculates the grade — nothing stops other unrelated code from accidentally changing that data.

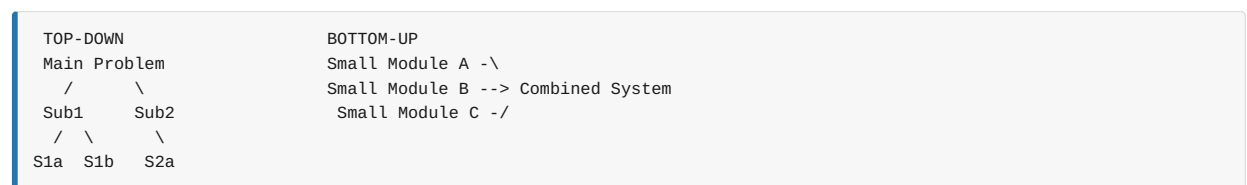
OOP fixes this by bundling data + the functions that act on that data into one single unit called an *object*. Think of a real-world “Student” — a student *has* data (name, roll number, marks) and can *do* things (calculate grade, print report card). OOP models exactly this: a **class** is the blueprint (“what a Student looks like in general”), and an **object** is one actual student built from that blueprint (e.g., “Ravi”, “Priya” — each with their own data, but sharing the same structure/behaviour).



This is why OOP gives better **data security** (a concept called *encapsulation* — data is wrapped safely inside the object) and better **reusability** (once the “Student” class/blueprint is built, unlimited student objects can be created from it without rewriting code).

4.3 Top-Down and Bottom-Up Design Approaches

- **Top-Down Design:** Start with the main problem, and break it down into smaller sub-problems/modules step by step (like decomposing a big task into smaller tasks).
- **Bottom-Up Design:** Start by building small basic modules first, then combine them to build the complete system.



Real-Life Analogy: Top-down is like planning a wedding by first deciding the overall event, then breaking it into catering, decoration, guest-list (sub-tasks). Bottom-up is like first preparing individual dishes and decorations separately, then assembling the whole event from these ready parts.

4.4 Characteristics of a Good Program

- Correctness (produces correct output), Readability, Efficiency (uses less time/memory), Portability (runs on different systems), Modularity, Well-documented, Robustness (handles errors gracefully), User-friendliness.

Key Takeaways: Structured = function-based; OOP = object-based; Top-down = break big into small; Bottom-up = build small then combine.

Exam-Oriented Notes: OOP vs Structured Programming difference table and Top-down vs Bottom-up are common 2/5-mark questions.

Important Terms: Module, Object, Class, Encapsulation, Modularity, Portability.

5. Programming Languages

5.1 Low-Level vs High-Level Languages

Feature	Low-Level Language	High-Level Language
Closeness	Close to machine (hardware)	Close to human (English-like)
Examples	Machine language, Assembly language	Python, Java, C++
Speed of execution	Very fast	Comparatively slower (needs translation)
Ease of writing	Difficult, machine-dependent	Easy, machine-independent

5.2 Generations of Programming Languages

Generation	Language Type	Example
1GL	Machine Language (binary 0s and 1s)	10110000
2GL	Assembly Language (mnemonics like ADD, MOV)	MOV A, B

3GL	High-level procedural languages	C, Pascal, BASIC
4GL	Very high-level, closer to natural language, often used for database queries	SQL
5GL	Languages based on solving problems using constraints/AI logic	Prolog

Real-Life Analogy: Machine language is like communicating in raw electrical signals (only the machine understands directly); High-level language is like speaking in plain English, which then needs an interpreter/translator to be understood by the machine.

Key Takeaways: Generation increases → language becomes easier for humans, harder/slower for machine to run directly (needs translation).

Exam-Oriented Notes: The generation table (1GL–5GL) with examples is a common 5-mark question.

6. Translators: Assembler, Compiler, Interpreter

Simple Definition: A translator converts a program written in a high-level/assembly language into machine language (binary) that the computer can execute.

Translator	Converts	How it Works	Example
Assembler	Assembly language → Machine language	Translates mnemonics directly to machine code	Converts MOV, ADD instructions
Compiler	High-level language → Machine language	Translates the entire program at once , then executes; reports all errors together after full translation	C, C++ compilers
Interpreter	High-level language → Machine language	Translates and executes line by line ; stops at the first error	Python interpreter

Comparison: Compiler vs Interpreter

Feature	Compiler	Interpreter
Translation	Whole program at once	Line by line
Speed of execution	Faster (already translated)	Slower (translates every run)
Error reporting	All errors shown after full compilation	Stops at first error
Output	Creates a separate executable file	No separate executable file created

Real-Life Analogy: A Compiler is like translating an entire book into another language before handing it over; an Interpreter is like a live interpreter translating a speech sentence-by-sentence as the speaker talks.

Deeper Concept — Why does this distinction matter? Think about *when* the translation happens relative to *when* the program runs: - With a **compiler**, translation and execution are two **separate stages**. First `gcc program.c -o program` creates `program.exe` (translation stage). Later, running `./program` is a completely separate step (execution stage) — and you can run it a thousand times without re-translating. - With an **interpreter**, translation and execution happen **together, one line at a time**, every single time you run the program. This is why Python programs don't produce a standalone `.exe` by default — the interpreter is required every time.

This explains two things students often get wrong: 1. **Why compiled programs run faster:** the CPU only has to execute ready-made machine code — no translation overhead during execution. 2. **Why interpreted languages are easier to debug/test quickly:** you can run a line, see the result, fix it, and re-run instantly — no separate compile step.

Note: Python (used in Units III–IV) technically uses a **hybrid approach** — it first compiles source code into an intermediate form called **bytecode** (`.pyc`), and then the **Python Virtual Machine (PVM)** interprets this bytecode line by line. For exam purposes at this level, it is enough to classify Python as an **interpreted language**.

Linker, Loader, Editors, IDE

- **Linker:** Combines multiple compiled code files (object files) and required libraries into a single executable program.
- **Loader:** Loads the executable program from secondary storage into main memory (RAM) so the CPU can execute it.
- **Editor:** A software tool used to write and edit program source code (e.g., Notepad, VS Code editor pane).
- **IDE (Integrated Development Environment):** A single software package that combines editor, compiler/interpreter, and debugger together for convenient program development (e.g., PyCharm, VS Code, Jupyter Notebook).

Key Takeaways: Assembler works on assembly code; Compiler translates all-at-once; Interpreter translates line-by-line; Linker combines files; Loader loads program into memory; IDE bundles all development tools together.

Exam-Oriented Notes: Compiler vs Interpreter is one of the **most frequently asked** questions (2/5 marks) in this unit.

Important Terms: Assembler, Compiler, Interpreter, Linker, Loader, Editor, IDE, Object file, Executable file.

7. Programming Errors

Simple Definition: Errors (also called “bugs”) are mistakes in a program that prevent it from working correctly.

Error Type	Description	Example
Syntax Error	Violates the grammar/rules of the language	Missing colon in <code>if x > 5</code>
Semantic Error	Statement is grammatically correct but meaning is wrong	Using <code>+</code> instead of <code>*</code> by logical mistake
Logical Error	Program runs but gives wrong output due to flawed logic	Wrong formula used in calculation
Compile-time Error	Detected by the compiler before execution (usually syntax errors)	Missing bracket
Run-time Error	Occurs during program execution	Division by zero
Link-time Error	Occurs when linker cannot find a referenced function/library	Undefined reference to a function
Environmental Error	Caused by the system environment (not the code itself)	Missing required file/library on system
Input/Output Error	Errors related to reading/writing data	File not found while opening a file

Testing and Debugging - Testing: The process of running a program to find whether it produces correct results and to discover errors. - **Debugging:** The process of locating and fixing (removing) the errors (bugs) found during testing.

Real-Life Analogy: Testing is like a doctor diagnosing symptoms (finding what’s wrong); Debugging is like the doctor treating and curing the illness (fixing the problem).

Key Takeaways: Syntax error = grammar mistake (caught before running); Logical error = wrong result despite correct grammar (hardest to catch); Run-time error = crash during execution.

Common Mistakes: Confusing Syntax error (grammar) with Logical error (wrong logic, but program runs fine).

Exam-Oriented Notes: Types of errors with one example each is a common 5-mark question. Testing vs Debugging difference is a common 2-mark question.

Important Terms: Bug, Syntax error, Semantic error, Logical error, Run-time error, Debugging.

★ Unit II Quick Revision

- Program development steps: Problem Analysis → Design → Coding → Testing → Debugging.
- Algorithm = step-by-step logic (language independent); Pseudocode = English-like structured form of algorithm.
- Control structures: Sequence, Selection, Iteration.
- Flowchart symbols: Oval=Start/Stop, Parallelogram=I/O, Rectangle=Process, Diamond=Decision, Arrow=Flow.
- Structured Programming (function-based) vs OOP (object-based).
- Top-Down (big → small) vs Bottom-Up (small → big) design.
- Low-level (machine-close) vs High-level (human-close) languages; 1GL to 5GL generations.
- Assembler (assembly → machine), Compiler (all at once), Interpreter (line by line).
- Linker (combines files), Loader (loads into RAM), IDE (editor+compiler+debugger together).
- Error types: Syntax, Semantic, Logical, Compile-time, Run-time, Link-time, Environmental, I/O errors.
- Testing = finding errors; Debugging = fixing errors.

? Unit II Important Questions

◆ 1 Mark Questions

- ✖ Define an algorithm.
- ✖ What is pseudocode?
- ✖ What is a bug?
- ✖ Expand IDE.
- 📖 What is the function of a Loader?
- ✖ Name the three control structures.

◆ 2 Mark Questions

- ★ Differentiate between Compiler and Interpreter.
- 📖 Differentiate between Syntax error and Logical error.
- 📖 Differentiate between Testing and Debugging.
- 📖 Differentiate between Top-Down and Bottom-Up approach.

- 🐞 What is the difference between Low-level and High-level languages?
- ⚡ What is the role of a Linker?

🔁 5 Mark Questions

- ⭐ Draw a flowchart to find the largest of three numbers.
- ⭐ Write an algorithm and pseudocode to find the factorial of a number.
- 🐞 Explain the different types of programming errors with examples.
- 🐞 Explain the generations of programming languages with examples.
- 🐞 Explain Structured Programming and Object-Oriented Programming with a comparison.

🔁 8 Mark Questions

- ⭐ Explain in detail the steps involved in program development with a diagram.
- ⭐ Explain Assembler, Compiler, and Interpreter in detail with a comparison table.
- 🐞 Explain flowchart symbols in detail and draw a flowchart for checking whether a number is prime.
- 🐞 Explain Top-Down and Bottom-Up design approaches with diagrams and examples.
- 🐞 Explain all types of programming errors (syntax, semantic, logical, compile-time, run-time, link-time, environmental, I/O) with examples for each.

UNIT III – Python Basics, Strings, Console I/O, and Control Instructions

1. Introduction to Python & Getting Started

Simple Definition: Python is a high-level, easy-to-read, interpreted programming language used for general-purpose programming.

Why Python is Popular: Simple, English-like syntax; free and open-source; huge standard library; interpreted (line-by-line execution, easy to debug); used in web development, data science, AI, automation, scripting.

Getting Started: Python code can be run by installing the Python interpreter and either: 1. Typing commands directly in the **interactive shell** (immediate output), or 2. Writing code in a `.py` file and running it as a **script**.

```
# my_first_program.py
print("Hello, World!")
```

Running this file executes it top to bottom through the Python interpreter.

Exam-Oriented Notes: Be ready to state 3–4 features of Python as a 2-mark question.

2. Python Basics

2.1 Identifiers

Definition: Names given to variables, functions, classes, etc. in a program. **Rules:** - Can contain letters, digits, and underscore (`_`). - Must **not** start with a digit. - Cannot be a Python keyword. - Case-sensitive (`Sum` and `sum` are different).

Valid: `age`, `_name`, `student1` **Invalid:** `1name`, `class` (keyword), `my-name` (hyphen not allowed)

2.2 Keywords

Definition: Reserved words that have a special, predefined meaning in Python and cannot be used as identifiers. **Examples:** `if`, `else`, `while`, `for`, `def`, `class`, `True`, `False`, `None`, `import`, `return`, `break`, `continue`, `pass`, `and`, `or`, `not`.

2.3 Python Types / Basic Types

Type	Description	Example
<code>int</code>	Whole numbers	<code>10</code> , <code>-5</code>
<code>float</code>	Decimal numbers	<code>3.14</code> , <code>-0.5</code>
<code>complex</code>	Numbers with real & imaginary part	<code>2+3j</code>
<code>str</code>	Text/string	<code>"Hello"</code>
<code>bool</code>	Boolean (True/False)	<code>True</code>
<code>NoneType</code>	Represents absence of value	<code>None</code>

2.4 Mutable and Immutable Types

- **Immutable:** Value **cannot** be changed after creation (a new object is created instead). Examples: `int`, `float`, `str`, `tuple`, `bool`.
- **Mutable:** Value **can** be changed in place after creation. Examples: `list`, `dict`, `set`.

Example:

```
x = "hello"
x = x + " world"  # a NEW string object is created; old one is discarded (immutable)

lst = [1, 2, 3]
lst.append(4)     # SAME list object is modified in place (mutable)
```

Real-Life Analogy: Immutable is like a printed photograph — to “change” it you must print a brand-new one. Mutable is like a whiteboard — you erase and rewrite on the same board.

Deeper Concept — proving it with `id()`: Every object in Python has a unique memory identity, which you can check using the built-in `id()` function. This is the clearest way to see mutability in action rather than just memorise the rule.


```

x = "hello"
print(id(x))      # e.g. 140312837
x = x + " world"
print(id(x))      # DIFFERENT id -> a brand-new string object was created

lst = [1, 2, 3]
print(id(lst))    # e.g. 140987654
lst.append(4)
print(id(lst))    # SAME id -> the original list object was modified in place

```

Why this matters practically: because mutable objects can change “in place,” if you assign a mutable object to a second variable, both variables point to the **same underlying object** — a change through one variable is visible through the other. Immutable objects don’t have this problem, because any “change” simply creates a brand-new object.

```

a = [1, 2, 3]
b = a              # b now refers to the SAME list object as a
b.append(4)
print(a)           # [1, 2, 3, 4] -> a changed too! (mutable, shared reference)

s1 = "hi"
s2 = s1
s2 = s2 + "!"
print(s1)          # "hi" -> unchanged (immutable, s2 now points to a new object)

```

Key Exam Takeaway: Mutable = same object modified in place (shared changes possible). Immutable = “changing” always produces a new object (no shared-change surprises).

2.5 Integer & Float Ranges

Python’s `int` type has **no fixed maximum limit** — it can grow as large as available memory allows (unlike languages like C which have fixed integer sizes). `float` follows the IEEE 754 double-precision standard, giving a very large range with limited decimal precision (about 15–17 significant digits).

2.6 Variable Type & Assignment

Python is **dynamically typed** — you don’t declare a variable’s type explicitly; it is decided automatically based on the value assigned.

```

x = 10             # x is int
x = "hello"        # now x becomes str (Python allows re-assignment to a different type)
a, b, c = 1, 2, 3  # multiple assignment
a = b = c = 0      # same value to multiple variables

```

2.7 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division (true division, gives float)	5 / 2	2.5
//	Floor Division (drops decimal part)	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponent (power)	5 ** 2	25

2.8 Precedence & Associativity

Precedence decides which operator is evaluated first when multiple operators appear together. **Associativity** decides the order when operators of the *same* precedence appear (left-to-right or right-to-left).

Order (high to low): `()` → `**` → `*`, `/`, `//`, `%` → `+`, `-`

```

result = 2 + 3 * 4    # * has higher precedence -> 2 + 12 = 14
result = 2 ** 3 ** 2   # ** is right-associative -> 2 ** (3 ** 2) = 2 ** 9 = 512

```

Deeper Concept — why associativity is needed at all: Precedence alone tells you *which operator wins* when different operators compete (e.g., `*` beats `+`). But it does **not** tell you what to do when the **same** operator appears more than once in a row — that’s exactly what associativity handles.

- `**` is evaluated **right-to-left**: `2 ** 3 ** 2` groups as `2 ** (3 ** 2)`, i.e. `2 ** 9 = 512`, **not** `(2 ** 3) ** 2 = 64`. This trips up almost every beginner.
- `+`, `-`, `*`, `/` are evaluated **left-to-right**: `20 - 5 - 3` groups as `(20 - 5) - 3 = 12`, not `20 - (5 - 3) = 18`.

Full precedence table (high → low), the exam-ready version:

Priority	Operators	Associativity
1 (highest)	<code>()</code> — parentheses	—
2	<code>**</code> — exponent	Right to Left
3	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Left to Right
4	<code>+</code> , <code>-</code> (binary)	Left to Right
5 (lowest, of these)	comparison (<code><</code> , <code>></code> , <code>==</code>)	Left to Right

Trick to remember: think of BODMAS/PEMDAS from school — **B**rackets → **O**rder/Exponent → **D**ivision/Multiplication → **A**ddition/Subtraction — Python follows the same core logic, plus the extra right-to-left rule for `**`.

2.9 Conversions (Type Casting)

Converting one data type into another.

```
int("10")      # str -> int => 10
float(10)       # int -> float => 10.0
str(10)         # int -> str => "10"
int(3.9)        # float -> int (truncates, does NOT round) => 3
```

2.10 Built-in Functions

Ready-made functions provided by Python that can be used directly without defining them. **Examples:** `print()`, `input()`, `len()`, `type()`, `int()`, `float()`, `str()`, `range()`, `sum()`, `max()`, `min()`, `sorted()`, `abs()`.

```
print(len("hello")) # 5
print(type(10))     # <class 'int'>
print(max(3, 9, 2)) # 9
```

2.11 Modules

A module is a file containing Python code (functions, variables) that can be reused by importing it into another program.

```
import math
print(math.sqrt(16)) # 4.0
print(math.pi)      # 3.14159...

from math import sqrt
print(sqrt(25))      # 5.0
```

2.12 Container Types

Data types that can hold multiple values together: `list`, `tuple`, `set`, `dict`, `str` (covered in detail: Strings below, and Lists/Sets/Tuples/Dictionaries in Unit IV).

2.13 Comments & Indentation

- **Comments:** Lines ignored by the interpreter, used to explain code.

```
# This is a single-line comment
"""
This is a
multi-line comment (docstring)
"""
```

- **Indentation:** Python uses **indentation (spaces)** instead of curly braces `{ }` to define blocks of code (like inside `if`, `for`, functions). Incorrect indentation causes an `IndentationError`.

```
if 5 > 2:
    print("Five is greater") # this line MUST be indented
```

2.14 Multi-lining

Writing one logical statement across multiple physical lines.

```
total = 1 + 2 + \
```

```

3 + 4          # using backslash

fruits = ["apple",
          "banana",
          "cherry"]  # allowed automatically inside brackets ()[]{}

```

2.15 Classes & Objects, Multiple Objects

Class: A blueprint/template for creating objects, defining what properties (data) and behaviours (functions) the objects will have. **Object:** An actual instance created from a class.

```

class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name, self.age)

s1 = Student("Asha", 20)  # object 1
s2 = Student("Ravi", 21)  # object 2 (multiple objects from same class)
s1.display()             # Asha 20
s2.display()             # Ravi 21

```

Real-Life Analogy: A class is like a cookie-cutter (design/shape), and objects are the individual cookies made using it — each cookie (object) can have its own toppings (data) but shares the same basic shape (structure).

Key Takeaways (Section 2): - Identifiers name things; Keywords are reserved and cannot be used as identifiers. - Mutable = changeable in place (list, dict, set); Immutable = cannot change in place (int, str, tuple). - `/` always gives float, `//` gives floor (integer-like) division, `%` gives remainder. - `**` is right-associative. - Indentation is **mandatory** in Python (not optional like in C/Java).

Common Mistakes - Using `=` (assignment) where `==` (comparison) was intended. - Forgetting that `/` gives a float even when dividing two integers evenly (`10/2` → `5.0`, not `5`). - Inconsistent indentation (mixing tabs and spaces) causing errors. - Believing strings are mutable (they are NOT — they are immutable).

Exam-Oriented Notes: Mutable vs Immutable, and Operator precedence table, are extremely common exam questions. Class & Object example programs are common 5/8-mark questions.

Important Terms: Identifier, Keyword, Dynamic typing, Type casting, Module, Docstring, Indentation, Class, Object, Constructor (`__init__`), `self`.

3. Strings

Simple Definition: A string is a sequence of characters enclosed in single (`' '`), double (`" "`), or triple (`''' '''`) quotes.

3.1 Accessing String Elements (Indexing & Slicing)

```

s = "PYTHON"
#   P Y T H O N
#   0 1 2 3 4 5      (positive indexing, left to right)
#  -6 -5 -4 -3 -2 -1  (negative indexing, right to left)

print(s[0])          # P
print(s[-1])         # N
print(s[1:4])         # YTH (slicing: start included, end excluded)
print(s[:3])          # PYT
print(s[::-1])        # NOHTYP (reversed string)

```

3.2 Properties

- Strings are **immutable** — once created, individual characters cannot be changed directly (`s[0] = 'X'` gives an error).
- Strings support indexing, slicing, and iteration (looping character by character).

3.3 Built-in Functions & Methods

Function/Method	Purpose	Example	Result
<code>len(s)</code>	Length of string	<code>len("hello")</code>	5
<code>s.upper()</code>	Convert to uppercase	<code>"hi".upper()</code>	"HI"
<code>s.lower()</code>	Convert to lowercase	<code>"HI".lower()</code>	"hi"
<code>s.strip()</code>	Remove leading/trailing spaces	<code>" hi ".strip()</code>	"hi"
<code>s.split(sep)</code>	Split into a list	<code>"a,b".split(",")</code>	['a', 'b']

<code>s.join(list)</code>	Join list into a string	<code>"-".join(['a', 'b'])</code>	<code>"a-b"</code>
<code>s.replace(a,b)</code>	Replace substring	<code>"hi".replace("h", "H")</code>	<code>"Hi"</code>
<code>s.find(sub)</code>	Index of first occurrence (-1 if not found)	<code>"hello".find("l")</code>	<code>2</code>
<code>s.count(sub)</code>	Count occurrences	<code>"hello".count("l")</code>	<code>2</code>
<code>s.startswith()/endswith()</code>	Check start/end match	<code>"hi".startswith("h")</code>	<code>True</code>
<code>s.isalpha()/isdigit()</code>	Check character types	<code>"123".isdigit()</code>	<code>True</code>

3.4 Conversions

```
str(123)          # int -> str => "123"
int("123")        # str -> int => 123 (fails if not a valid number)
list("abc")       # str -> list => ['a', 'b', 'c']
```

3.5 Comparisons

Strings are compared **lexicographically** (dictionary order, based on ASCII/Unicode values of characters).

```
print("apple" == "apple") # True
print("apple" < "banana") # True (because 'a' < 'b')
```

Real-Life Analogy: A string is like a chain of beads (characters) — each bead has a fixed position (index), and you can look at any bead or a section of the chain (slice), but you cannot swap out a single bead in the same chain (immutable) — you’d need to make a whole new chain.

Key Takeaways: Indexing starts at 0; slicing excludes the end index; strings are immutable; string comparison is lexicographic.

Common Mistakes: Trying to modify a string directly (`s[0]='A' → Error`); confusing `s.find()` (returns -1 if not found) with `s.index()` (raises an error if not found).

Exam-Oriented Notes: String slicing and built-in string methods (with sample outputs) are very frequently asked as 5-mark practical questions.

Important Terms: Indexing, Slicing, Immutable, Lexicographic comparison, Method.

4. Console I/O

Simple Definition: Console I/O refers to taking input from and displaying output to the user through the command-line console/terminal.

4.1 I/O Operations

```
name = input("Enter your name: ") # input() always returns a string
print("Hello,", name)
age = int(input("Enter your age: ")) # convert input string to int explicitly
```

4.2 Formatted Printing

```
name, age = "Asha", 20
print("Name: %s, Age: %d" % (name, age)) # % formatting (old style)
print("Name: {}, Age: {}".format(name, age)) # .format() method
print(f"Name: {name}, Age: {age}") # f-string (modern, preferred)
```

Key Takeaways: `input()` always returns a string, so numeric input must be explicitly type-cast using `int()` / `float()`. f-strings are the most modern and readable way to format output.

Exam-Oriented Notes: Writing a program using `input()` + f-string formatting is a common 2/5-mark practical question.

5. Decision Control Instruction

Simple Definition: Statements that allow a program to choose between different paths of execution based on a condition.

5.1 Logical Operators

Operator	Meaning	Example

and	True if both conditions are True	(5>2) and (3>1) → True
or	True if at least one condition is True	(5>2) or (1>3) → True
not	Reverses the boolean value	not(5>2) → False

5.2 Conditional Expressions (if / elif / else)

```
age = 20
if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

Conditional (Ternary) Expression:

```
status = "Adult" if age >= 18 else "Minor"
```

5.3 all() and any()

- all(iterable) : Returns True if **all** elements are truthy (or the iterable is empty).
- any(iterable) : Returns True if **at least one** element is truthy.

```
print(all([True, True, False])) # False
print(any([False, False, True])) # True
```

5.4 Receiving Input (in the context of decisions)

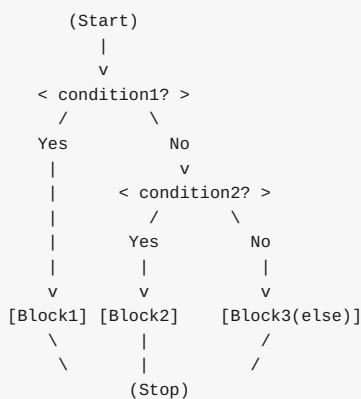
```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

5.5 pass Statement

pass is a **null operation** — it does nothing. Used as a placeholder where a statement is syntactically required but no action is needed yet.

```
if True:
    pass # will implement this later, no error meanwhile
```

Flowchart of if-elif-else:



Key Takeaways: elif avoids writing multiple nested if statements; pass is a placeholder, not a “do-nothing forever” — it just means “no operation here.”

Common Mistakes: Using = instead of == in a condition; forgetting the colon : at the end of if / elif / else lines; incorrect indentation of the block under if.

Exam-Oriented Notes: all() / any() and pass statement usage are commonly tested as 1/2-mark questions; if-elif-else programs are common 5-mark questions.

Important Terms: Condition, Boolean expression, Ternary/Conditional expression, pass.

6. Repetition Control Instruction (Loops)

Simple Definition: Loops are used to execute a block of statements repeatedly until a condition is satisfied.

6.1 Types of Loops

for loop: Used when the number of iterations is known/based on a sequence.

```
for i in range(1, 6):    # 1,2,3,4,5 (5 excluded)
    print(i)
```

while loop: Used when repetition depends on a condition being true (number of iterations may be unknown in advance).

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

6.2 break and continue

- **break** : Immediately exits/terminates the loop.
- **continue** : Skips the rest of the current iteration and moves to the next one.

```
for i in range(1, 10):
    if i == 5:
        break          # loop stops completely at 5
    print(i)

for i in range(1, 6):
    if i == 3:
        continue       # skips printing 3, continues loop
    print(i)
```

6.3 else Block of a Loop

Python loops can have an **else** block, which executes **only if the loop completes normally (without hitting a break)**.

```
for i in range(1, 5):
    print(i)
else:
    print("Loop finished without break")
```

Comparison: for vs while

Feature	for loop	while loop
Use case	Known number of iterations / iterating a sequence	Condition-based, unknown number of iterations
Example	for i in range(5):	while x < 5:

Real-Life Analogy: A **for** loop is like reading every page of a book you already have (known, countable pages). A **while** loop is like eating food “while you’re still hungry” — you don’t know exactly how many bites it’ll take.

Key Takeaways: **break** = stop loop entirely; **continue** = skip current iteration only; loop’s **else** runs only if no **break** occurred.

Common Mistakes: Creating an infinite loop by forgetting to update the loop variable in a **while** loop; confusing **break** and **continue**.

Exam-Oriented Notes: Programs using loops (printing patterns, sum of series, prime check) are very common 5/8-mark questions. **break** vs **continue** is a common 2-mark question.

Important Terms: Iteration, `range()`, **break**, **continue**, Infinite loop, Loop **else**.

★ Unit III Quick Revision

- Identifiers: letters/digits/underscore, cannot start with digit, cannot be a keyword.
- Immutable: int, float, str, tuple, bool | Mutable: list, dict, set.
- Arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`; `/` → float, `//` → floor division.
- Precedence: `()` > `**` > `*` / `//` > `+` - > `%`; `**` is right-associative.
- Type conversion functions: `int()`, `float()`, `str()`.
- Modules imported using `import` / `from ... import ...`.

- Comments: `#` single line, `''' '''` multi-line; Indentation defines blocks (mandatory).
- Class = blueprint, Object = instance; `self` refers to the current object.
- Strings: immutable, 0-based indexing, negative indexing from -1, slicing `[start:end:step]`.
- Common string methods: `upper()`, `lower()`, `split()`, `join()`, `replace()`, `find()`, `strip()`.
- `input()` always returns a string — must cast for numbers.
- f-strings: `f"{variable}"` — modern formatted printing.
- Logical operators: `and`, `or`, `not`; `all()` / `any()` on iterables.
- `pass` = do-nothing placeholder statement.
- `for` loop = known iterations; `while` loop = condition-based.
- `break` = exit loop; `continue` = skip to next iteration; loop `else` runs only if no break occurred.

? Unit III Important Questions

◆ 1 Mark Questions

- ✖ What is an identifier? Give an example.
- ✖ Name any five Python keywords.
- ✖ What does the `input()` function return by default?
- ✖ What is the use of the `pass` statement?
- 🐼 Give an example each of a mutable and an immutable data type.
- ✖ What is the output of `5 // 2` and `5 / 2`?

◆ 2 Mark Questions

- ✖ Differentiate between mutable and immutable data types with examples.
- ✖ Differentiate between `break` and `continue`.
- 🐼 Differentiate between `for` loop and `while` loop.
- 🐼 What is the difference between `/` and `//` operators?
- 🐼 What is string slicing? Give an example.
- ✖ What is the difference between `find()` and `index()` string methods?

◆ 5 Mark Questions

- ✖ Explain operator precedence and associativity in Python with an example.
- ✖ Write a Python program using a class with a constructor to create and display multiple student objects.
- 🐼 Explain any five built-in string methods with examples.
- 🐼 Write a Python program to demonstrate `if-elif-else` to classify a number as positive, negative, or zero.
- 🐼 Explain `all()` and `any()` functions with examples.

◆ 8 Mark Questions

- ✖ Explain Python identifiers, keywords, variable assignment, and data types in detail with examples.
- ✖ Explain string indexing, slicing, and at least eight built-in string methods with examples and outputs.
- 🐼 Explain decision control statements (`if`, `elif`, `else`, ternary expression) with programs and flowcharts.
- 🐼 Explain repetition control statements (`for`, `while`, `break`, `continue`, loop `else`) with programs.
- 🐼 Explain the concept of classes and objects in Python with a complete program creating multiple objects.

UNIT IV – Data Structures, Comprehensions, and Functions

1. Lists

Simple Definition: A list is an **ordered, mutable** collection of items, which can be of different data types, written inside square brackets `[]`.

Creating:

```
lst = [10, 20, 30, "hello", 3.5]
empty = []
```

Accessing (Indexing/Slicing):

```
print(lst[0])    # 10
print(lst[-1])   # 3.5
print(lst[1:3])  # [20, 30]
```

Looping:

```
for item in lst:
    print(item)
```

Basic Operations & Built-in Functions/Methods: | Operation | Example | Result/Effect | |---| | Add item at end | `lst.append(40)` | adds 40 at the end | | Insert at position | `lst.insert(1, 99)` | inserts 99 at index 1 | | Remove by value | `lst.remove(20)` | removes first occurrence of 20 | | Remove by index | `lst.pop(0)` | removes & returns item at index 0 | | Sort | `lst.sort()` | sorts list in place | | Reverse | `lst.reverse()` | reverses list in place | | Length | `len(lst)` | number of elements | | Concatenation | `lst1 + lst2` | joins two lists | | Repetition | `lst * 2` | repeats list twice | | Membership | `20 in lst` | True / False |

Data-structure operations using a list: - **Stack (LIFO – Last In First Out):** use `append()` to push, `pop()` to pop (removes from the end). - **Queue (FIFO – First In First Out):** use `append()` to enqueue, `pop(0)` to dequeue (removes from the front).

```
# Stack
stack = []
stack.append(1); stack.append(2); stack.append(3)
print(stack.pop()) # 3 (Last In, First Out)

# Queue
queue = []
queue.append(1); queue.append(2); queue.append(3)
print(queue.pop(0)) # 1 (First In, First Out)
```

Real-Life Analogy: A Stack is like a pile of plates — you add/remove from the top only. A Queue is like a line at a ticket counter — first person in line is served (removed) first.

Key Takeaways: Lists are mutable and ordered; allow duplicate values; support mixed data types.

Exam-Oriented Notes: List-based Stack/Queue implementation is a very common lab/8-mark question.

2. Sets

Simple Definition: A set is an **unordered, mutable** collection of **unique** (no duplicate) items, written inside curly braces `{}`.

Creating:

```
s = {1, 2, 3, 3, 2} # duplicates automatically removed -> {1, 2, 3}
empty_set = set()  # NOT {} (that creates an empty dict!)
```

Basic Operations: | Operation | Example | Meaning | |---| | Add | `s.add(4)` | adds an element | | Remove | `s.remove(2)` | removes element (error if absent) | | Discard | `s.discard(2)` | removes if present (no error if absent) | | Union | `s1 \ s2` | all elements from both sets | | Intersection | `s1 & s2` | common elements only | | Difference | `s1 - s2` | elements in s1 but not in s2 |

Real-Life Analogy: A set is like a basket of unique fruit types — even if you try to put two apples, only one “apple” entry exists in terms of unique labelling.

Key Takeaways: No duplicates, no indexing (unordered — cannot access by position); useful for membership tests and removing duplicates.

Common Mistakes: Using `{}` thinking it creates an empty set — it actually creates an empty **dictionary**.

3. Tuples

Simple Definition: A tuple is an **ordered, immutable** collection of items, written inside round brackets `()`.

Creating:

```
t = (10, 20, 30)
single = (5,)      # comma needed for a single-element tuple
```

Basic Operations:

```
print(t[0])          # 10 (indexing works, like lists)
print(t[1:])         # (20, 30) (slicing works)
print(len(t))        # 3
print(t.count(10))   # 1
print(t.index(20))   # 1
new_t = t + (40,)    # concatenation creates a NEW tuple (immutable)
```

Comparison: List vs Tuple

Feature	List	Tuple
Brackets	[]	()
Mutability	Mutable	Immutable
Speed	Slower	Faster (due to immutability)
Use case	Data that may change	Fixed/constant data (e.g., coordinates)

Real-Life Analogy: A tuple is like a sealed, printed certificate — once issued, it cannot be edited. A list is like a to-do list on a whiteboard — items can be added, removed or changed anytime.

Key Takeaways: Tuples cannot be modified after creation; they are commonly used for fixed collections of related values.

4. Dictionaries

Simple Definition: A dictionary is an **unordered (insertion-ordered since Python 3.7), mutable** collection of **key-value pairs**, written inside curly braces `{}`.

Creating:

```
student = {"name": "Asha", "roll": 101, "age": 20}
```

Accessing, Looping, Basic Operations:

```
print(student["name"])      # Asha
print(student.get("roll"))  # 101 (safer - returns None instead of error if key absent)
student["age"] = 21        # update value
student["marks"] = 90      # add new key-value pair
del student["marks"]       # remove a key

for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, "->", value)

print(student.keys())      # dict_keys(['name', 'roll', 'age'])
print(student.values())    # dict_values(['Asha', 101, 21])
```

Real-Life Analogy: A dictionary (data type) works exactly like a real dictionary book — you look up a **word (key)** to instantly get its **meaning (value)**.

Key Takeaways: Keys must be unique and immutable (e.g., string, int, tuple); values can be of any type and can repeat.

Common Mistakes: Using a mutable type (like a list) as a dictionary key — this raises a `TypeError` since keys must be immutable/hashable.

Comparison Table: List vs Tuple vs Set vs Dictionary

Feature	List	Tuple	Set	Dictionary
Brackets	[]	()	{ }	{key: value}
Ordered	Yes	Yes	No	Insertion-ordered

Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique, values can repeat
Indexing	Yes	Yes	No	Access via key, not index

Exam-Oriented Notes: This comparison table (List/Tuple/Set/Dict) is one of the **most important tables in the entire syllabus** — expect it in 5 or 8 mark questions.

5. Comprehensions (List, Set, Dictionary)

Simple Definition: A comprehension is a short, one-line way to create a new list/set/dictionary by applying an expression to each item of an existing iterable, optionally with a condition.

List Comprehension:

```
squares = [x**2 for x in range(1, 6)]           # [1, 4, 9, 16, 25]
evens = [x for x in range(1, 11) if x % 2 == 0] # [2, 4, 6, 8, 10]
```

Set Comprehension:

```
unique_lengths = {len(word) for word in ["hi", "bye", "ok"]} # {2, 3}
```

Dictionary Comprehension:

```
squares_dict = {x: x**2 for x in range(1, 5)} # {1:1, 2:4, 3:9, 4:16}
```

Concept Explanation (Basic Idea → Technical): Normally, to build a new list from an old one, you'd write a `for` loop with `append()` across 3 lines. A comprehension compresses that loop into a single, readable line: `[expression for item in iterable if condition]`.

Real-Life Analogy: A comprehension is like an assembly line that takes each raw item, applies one operation to it, optionally filters out unwanted ones, and drops it directly into the final container — all in one smooth motion instead of separate manual steps.

Key Takeaways: Comprehensions are shorter and often faster than manual loops for creating new collections. Syntax pattern is the same across list `[]`, set `{}`, and dict `{k:v}`.

Exam-Oriented Notes: Writing a comprehension for squares/evens/filtering is a very common 2/5-mark practical question.

6. Functions

Simple Definition: A function is a named, reusable block of code that performs a specific task and can be “called” (invoked) whenever needed.

6.1 Built-in vs User-Defined Functions

- **Built-in functions:** Already provided by Python (e.g., `len()`, `print()`, `type()`).
- **User-defined functions:** Created by the programmer using the `def` keyword.

```
def greet(name):           # function definition
    print("Hello,", name)

greet("Asha")              # function call (invoking)
```

6.2 Invoking Functions & Unpacking Arguments

```
def add(a, b):
    return a + b

print(add(3, 4))           # positional arguments -> 7
print(add(b=4, a=3))      # keyword arguments -> 7

def show(*args):           # *args collects extra positional args into a tuple
    print(args)
show(1, 2, 3)              # (1, 2, 3)

nums = [3, 4]
print(add(*nums))          # unpacking a list into positional arguments -> 7

def info(**kwargs):        # **kwargs collects extra keyword args into a dict
    print(kwargs)
```

```
info(name="Asha", age=20) # {'name': 'Asha', 'age': 20}
```

6.3 Recursive Function

Definition: A function that calls **itself** to solve a smaller version of the same problem, until it reaches a base case that stops the recursion.

```
def factorial(n):  
    if n == 0:                # base case (stops recursion)  
        return 1  
    else:  
        return n * factorial(n - 1)  # recursive call  
  
print(factorial(5))  # 120
```

How recursion works (Call Stack) for factorial(3):

```
factorial(3) = 3 * factorial(2)  
            = 3 * (2 * factorial(1))  
            = 3 * (2 * (1 * factorial(0)))  
            = 3 * (2 * (1 * 1))  
            = 6
```

Deeper Concept — how the computer actually keeps track: Every time a function calls itself, Python does **not** overwrite the previous call — it **pauses** it and pushes a fresh copy onto a structure called the **call stack**, waiting for the new (inner) call to finish first. Only once the base case is hit does the stack start “unwinding,” resolving each paused call one by one, from the innermost outward.

CALLING (stack builds up)	RETURNING (stack unwinds)
factorial(3) ↴	factorial(3) = 3*2 = 6 ←
factorial(2) ↴	factorial(2) = 2*1 = 2 ←
factorial(1) ↴	factorial(1) = 1*1 = 1 ←
factorial(0) → returns 1	
(base case reached, stack starts unwinding upward)	

Two things **MUST** be present for recursion to work correctly: 1. **Base Case:** a condition that stops further recursive calls (`if n == 0: return 1`) — without this, the function calls itself forever, eventually causing a `RecursionError` (stack overflow). 2. **Recursive Case:** the function must call itself with an input that moves **closer** to the base case each time (`n - 1` , not `n` or `n + 1`) — otherwise it never terminates.

Real-Life Analogy: Recursion is like Russian nesting dolls (Matryoshka) — to open the outermost doll, you must first open the one inside it, and that one requires opening the one inside *it*, and so on, until you reach the smallest doll (the base case) that doesn't open any further. Then you close them back up one by one, from smallest to largest.

6.4 Iteration vs Recursion

Feature	Iteration (loop)	Recursion
Mechanism	Repeats using <code>for</code> / <code>while</code>	Function calls itself
Memory use	Uses less memory	Uses more memory (call stack for each call)
Base/stop condition	Loop condition	Base case
Readability	Simple for straightforward repetition	More elegant for naturally recursive problems (e.g., tree traversal, factorial)

Real-Life Analogy: Recursion is like standing between two mirrors facing each other — each mirror reflects a smaller copy of the same image, until it becomes too small to see (the base case). Iteration is like walking down a straight numbered hallway, one step (loop cycle) at a time.

Key Takeaways: Every recursive function **MUST** have a base case, or it will run forever (causing a `RecursionError` /stack overflow). `*args` collects extra positional arguments as a tuple; `**kwargs` collects extra keyword arguments as a dictionary.

Common Mistakes: Forgetting the base case in recursion (infinite recursion); confusing `*args` / `**kwargs` unpacking direction (packing when defining a function vs unpacking when calling one).

Exam-Oriented Notes: Recursive factorial/power/Fibonacci functions and Iteration vs Recursion difference are very frequently asked as 5/8-mark questions.

Important Terms: Parameter, Argument, Return value, `*args` , `**kwargs` , Base case, Call stack, Recursion.

★ Unit IV Quick Revision

- List `[]` : ordered, mutable, allows duplicates; supports stack (LIFO, `append` / `pop`) & queue (FIFO, `append` / `pop(0)`).
- Set `{ }` : unordered, mutable, no duplicates; operations: union `|` , intersection `&` , difference `-` .
- Tuple `( )` : ordered, immutable, allows duplicates; faster than list.

- Dictionary `{key:value}` : mutable, keys unique & immutable, access via `get()` / `[]` , loop via `.items()` .
- Comprehension pattern: `[expr for item in iterable if condition]` — works for list `[]` , set `{}` , dict `{k:v}` .
- Function defined using `def ; *args` → extra positional args as tuple; `**kwargs` → extra keyword args as dict.
- Recursion = function calling itself + must have a base case; Iteration = loop-based repetition.
- Comparison table List vs Tuple vs Set vs Dictionary is a must-know for exams.

? Unit IV Important Questions

1 Mark Questions

- ✖ What symbol/brackets are used to create a list, tuple, set, and dictionary?
- ✖ What is a base case in recursion?
- ✖ What does `*args` collect in a function definition?
- 🐞 Can a dictionary have duplicate keys?
- ✖ Is a tuple mutable or immutable?

2 Mark Questions

- ✖ Differentiate between List and Tuple.
- ✖ Differentiate between Iteration and Recursion.
- 🐞 Differentiate between `*args` and `**kwargs` .
- 🐞 What is the difference between `remove()` and `discard()` in sets?
- 🐞 What is a dictionary comprehension? Give an example.

5 Mark Questions

- ✖ Write a Python program to implement Stack and Queue operations using a list.
- ✖ Write a recursive function to compute the factorial of a number, and explain how it works.
- 🐞 Explain basic operations on a dictionary with examples (add, update, delete, access).
- 🐞 Write list, set, and dictionary comprehensions to generate squares of numbers from 1 to 10.
- 🐞 Explain the union, intersection, and difference operations on sets with examples.

8 Mark Questions

- ✖ Compare List, Tuple, Set, and Dictionary in detail with a table and example code for each.
- ✖ Explain user-defined functions in Python: parameters, `*args` , `**kwargs` , and return values, with example programs.
- 🐞 Explain recursion with an example (factorial or Fibonacci), and compare it with the iterative approach.
- 🐞 Write a program to create a dictionary of N students (roll number as key) and search for a student by name; explain the logic.

LAB PROGRAMS – Programming Using Python (I-Lab)

Below are concise, working solutions for all 24 lab exercises listed in the syllabus. Each includes the code and a short explanation. These reinforce the concepts from Units III and IV.

1. Demonstrate various arithmetic operators

```
a, b = 15, 4
print("Add:", a + b)
print("Sub:", a - b)
print("Mul:", a * b)
print("Div:", a / b)
print("Floor Div:", a // b)
print("Modulus:", a % b)
print("Power:", a ** b)
```

Explanation: Applies each arithmetic operator on two numbers and prints the result.

2. Use trigonometric functions from the math module

```
import math
angle = math.radians(30) # convert degrees to radians
print("sin(30):", math.sin(angle))
print("cos(30):", math.cos(angle))
print("tan(30):", math.tan(angle))
```

3. Convert temperatures (Fahrenheit <-> Centigrade)

```
# a. Fahrenheit to Centigrade
f = float(input("Enter Fahrenheit: "))
c = (f - 32) * 5/9
print("Centigrade:", c)

# b. Centigrade to Fahrenheit
c2 = float(input("Enter Centigrade: "))
f2 = (c2 * 9/5) + 32
print("Fahrenheit:", f2)
```

4. Find the roots of a quadratic equation $ax^2 + bx + c = 0$

```
import math
a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))
d = b**2 - 4*a*c
if d > 0:
    r1 = (-b + math.sqrt(d)) / (2*a)
    r2 = (-b - math.sqrt(d)) / (2*a)
    print("Roots are real:", r1, r2)
elif d == 0:
    r1 = -b / (2*a)
    print("Roots are equal:", r1)
else:
    real = -b / (2*a)
    imag = math.sqrt(-d) / (2*a)
    print(f"Roots are complex: {real}+{imag}i, {real}-{imag}i")
```

5. Demonstrate various String functions

```
s = " Python Programming "
print(s.strip())
print(s.upper())
print(s.lower())
print(s.strip().split())
print(s.strip().replace("Python", "Java"))
print(len(s))
```

6. Enter name and perform operations

```
full_name = input("Enter your full name: ") # e.g. "Asha Kumari Sharma"
parts = full_name.split()
print("a. Middle name:", parts[1] if len(parts) > 2 else "No middle name")
print("b. Last name (slicing):", parts[-1])
print("c. Surname first:", parts[-1] + " " + " ".join(parts[:-1]))
```

7. Check whether an integer is even or odd

```
num = int(input("Enter an integer: "))
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

8. Find the youngest among Shyam, Dugu, and Ishan

```
shyam = int(input("Enter Shyam's age: "))
dugu = int(input("Enter Dugu's age: "))
ishan = int(input("Enter Ishan's age: "))
youngest = min(shyam, dugu, ishan)
if youngest == shyam:
    print("Shyam is the youngest")
elif youngest == dugu:
    print("Dugu is the youngest")
else:
    print("Ishan is the youngest")
```

9. Check if three points fall on one straight line

```
x1, y1 = map(float, input("Enter x1 y1: ").split())
x2, y2 = map(float, input("Enter x2 y2: ").split())
x3, y3 = map(float, input("Enter x3 y3: ").split())
# Points are collinear if slope(1,2) == slope(2,3), checked via cross-multiplication
if (y2 - y1) * (x3 - x2) == (y3 - y2) * (x2 - x1):
    print("Points are collinear (on one straight line)")
else:
    print("Points are NOT collinear")
```

10. Print odd numbers from n1 to n2

```
n1 = int(input("Enter n1: "))
n2 = int(input("Enter n2: "))
for i in range(n1, n2 + 1):
    if i % 2 != 0:
        print(i, end=" ")
```

11. Find the factorial of a number

```
n = int(input("Enter a number: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("Factorial:", fact)
```

12. Print all prime numbers between n1 and n2

```
n1 = int(input("Enter n1: "))
n2 = int(input("Enter n2: "))
for num in range(n1, n2 + 1):
    if num > 1:
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                break
        else:
            print(num, end=" ")
```

13. Basic operations on a list

```
lst = [10, 20, 30]
lst.append(40)
lst.insert(1, 15)
lst.remove(20)
print("List:", lst)
print("Length:", len(lst))
lst.sort()
print("Sorted:", lst)
```

14. Stack and Queue operations using a list

```
stack = []
```

```

stack.append(1); stack.append(2); stack.append(3)
print("Stack pop (LIFO):", stack.pop())    # 3

queue = []
queue.append(1); queue.append(2); queue.append(3)
print("Queue pop (FIFO):", queue.pop(0))    # 1

```

15. Ask data of five students (name, roll, age) and sort by roll number

```

students = []
for i in range(5):
    name = input("Name: ")
    roll = int(input("Roll No: "))
    age = int(input("Age: "))
    students.append([name, roll, age])

students.sort(key=lambda s: s[1])    # sort by roll number (index 1)
print("Sorted by roll number:")
for s in students:
    print(s)

```

16. Add two square matrices

```

n = int(input("Enter dimension n: "))
print("Enter Matrix A row by row:")
A = [list(map(int, input().split())) for _ in range(n)]
print("Enter Matrix B row by row:")
B = [list(map(int, input().split())) for _ in range(n)]

C = [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
print("Sum of Matrices:")
for row in C:
    print(row)

```

17. Basic operations on a tuple

```

t = (10, 20, 30, 20)
print("Length:", len(t))
print("Count of 20:", t.count(20))
print("Index of 30:", t.index(30))
new_t = t + (40,)
print("New tuple:", new_t)

```

18. Share portfolio using tuples

```

shares = []
n = int(input("Number of shares held: "))
for i in range(n):
    name = input("Share name: ")
    cost = float(input("Cost price: "))
    qty = int(input("Number of shares: "))
    sell = float(input("Selling price: "))
    shares.append((name, cost, qty, sell))

total_cost = sum(s[1] * s[2] for s in shares)
total_gain = sum((s[3] - s[1]) * s[2] for s in shares)
print("a. Total cost of portfolio:", total_cost)
print("b. Total amount gained/lost:", total_gain)

```

19. Basic operations on a set

```

s = {10, 20, 30}
s.add(40)
s.discard(20)
print("Set:", s)
print("Union with {50,60}:", s | {50, 60})
print("Intersection with {10,40,99}:", s & {10, 40, 99})

```

20. Set of student names — add, modify, delete

```

names = set()
for i in range(5):
    names.add(input(f"Enter name {i+1}: "))
print("Initial set:", names)

old = input("Enter name to modify: ")

```

```

new = input("Enter new name: ")
names.discard(old)
names.add(new)

d1 = input("Enter first name to delete: ")
d2 = input("Enter second name to delete: ")
names.discard(d1)
names.discard(d2)
print("Final set:", names)

```

21. Basic operations on a dictionary

```

d = {"a": 1, "b": 2}
d["c"] = 3          # add
d["a"] = 10         # update
del d["b"]          # delete
print("Dictionary:", d)
print("Keys:", list(d.keys()))
print("Values:", list(d.values()))

```

22. Dictionary of N students (roll number as key); search by name

```

n = int(input("Enter number of students: "))
students = {}
for i in range(n):
    roll = int(input("Roll number: "))
    name = input("Name: ")
    students[roll] = [name]

search_name = input("Enter name to search: ")
found = False
for roll, data in students.items():
    if data[0] == search_name:
        print("Found -> Roll:", roll, "Data:", data)
        found = True
        break
if not found:
    print("Student not found")

```

23. Basic comprehensions on list, set, and dictionary

```

squares_list = [x**2 for x in range(1, 6)]
squares_set = {x**2 for x in range(1, 6)}
squares_dict = {x: x**2 for x in range(1, 6)}
print("List comprehension:", squares_list)
print("Set comprehension:", squares_set)
print("Dict comprehension:", squares_dict)

```

24. Find x^n using a function (without using any math function)

```

def power(x, n):
    result = 1
    for i in range(n):
        result *= x
    return result

x = float(input("Enter x: "))
n = int(input("Enter n: "))
print(f"{x}^{n} =", power(x, n))

```


EXAM PREPARATION GUIDE (Full Syllabus)

1. Most Important Topics from the Entire Syllabus

- Block diagram of a computer & CPU components (Unit I)
- Memory hierarchy, RAM vs ROM (Unit I)
- Network types (LAN/MAN/WAN) & devices (Unit I)
- Algorithm, pseudocode, flowchart symbols (Unit II)
- Compiler vs Interpreter (Unit II) — **highly repeated**
- Programming errors: syntax vs logical vs run-time (Unit II)
- Mutable vs Immutable data types (Unit III)
- Operator precedence & associativity (Unit III)
- String slicing and string methods (Unit III)
- Decision control (`if-elif-else`) and loops (`for` , `while` , `break` , `continue`) (Unit III)
- Class & Object with constructor (Unit III)
- List vs Tuple vs Set vs Dictionary comparison (Unit IV) — **highly repeated**
- Stack & Queue using list (Unit IV)
- Comprehensions (list/set/dict) (Unit IV)
- Recursion vs Iteration, recursive factorial function (Unit IV)

2. Most Important Definitions

- Computer, Algorithm, Pseudocode, Flowchart, Compiler, Interpreter, Bug, Debugging, Identifier, Keyword, Mutable, Immutable, List, Tuple, Set, Dictionary, Function, Recursion.

3. Most Important Diagrams

- Block diagram of a computer (Input–CPU–Memory–Output–Storage).
- Memory hierarchy pyramid.
- Flowchart symbols and a sample flowchart (even/odd or largest of 3 numbers).
- Top-Down vs Bottom-Up design diagram.
- if-elif-else flowchart.

4. Most Important Formulas / Syntax

- Arithmetic operators & precedence order: `()` → `**` → `*` / `/` % → `+` - .
- String slicing: `s[start:end:step]` .
- List/Set/Dict comprehension: `[expr for item in iterable if condition]` .
- Recursive function structure: base case + recursive call.
- Quadratic roots formula: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

5. Most Important Differences (Tables to Revise)

- RAM vs ROM
- Compiler vs Interpreter ★ (very frequently asked)
- Syntax Error vs Logical Error
- Top-Down vs Bottom-Up Design
- List vs Tuple vs Set vs Dictionary ★ (very frequently asked)
- Mutable vs Immutable
- Iteration vs Recursion
- `break` vs `continue`
- `for` loop vs `while` loop

6. Most Repeated/Expected Question Types

- “Differentiate between X and Y” (2 marks) — appears in every unit.
- “Draw and explain the block diagram / flowchart / hierarchy” (5–8 marks).
- “Write a Python program to...” (5–8 marks, from lab list).
- “Explain the types of...” with a table (5–8 marks).
- “Explain with an example and diagram” (8 marks) — combines theory + code + visualization.

7. Unit-Wise Important Questions (Summary List)

- **Unit I:** Block diagram of computer, Memory hierarchy, RAM vs ROM, Network devices, Types of software.
- **Unit II:** Compiler vs Interpreter, Algorithm+Flowchart for a problem, Programming errors, Top-down vs Bottom-up.
- **Unit III:** Mutable vs Immutable, Operator precedence, String methods, if-elif-else program, Class & Object program.
- **Unit IV:** List vs Tuple vs Set vs Dictionary, Stack/Queue using list, Recursive factorial, Comprehensions.

8. Last-Minute Revision Notes

- Computer = Input+Process+Output+Storage; CPU = CU+ALU+Registers.
- Memory speed: Registers > Cache > RAM > Secondary Memory.
- Compiler = translates all at once; Interpreter = line by line.
- Python: `/` gives float, `//` gives floor division, `**` is right-associative.
- Strings & Tuples = immutable; Lists, Sets, Dicts = mutable.
- List `[]`, Tuple `()`, Set `{}` (unique), Dict `{key:value}`.
- Recursion needs a base case; `input()` returns a string always.
- `break` exits loop, `continue` skips current iteration.

9. Exam Strategy

1. Revise the **comparison tables** first (they cover multiple 1/2-mark questions in one shot).
2. Practice **drawing** the block diagram, memory hierarchy, and flowcharts by hand — diagrams often carry separate marks.
3. Practice writing **short Python programs by hand** (without an IDE) — most 5/8-mark practical questions expect handwritten code.
4. For 8-mark questions, structure your answer as: **Definition** → **Explanation** → **Diagram/Table** → **Example/Code** → **Key Points** — this format satisfies most university marking schemes.
5. Keep 1-mark definitions crisp (1–2 lines) — do not write paragraphs for 1-mark answers.
6. Attempt all “Differentiate between...” questions using a **table format** — it is faster to write and easier for examiners to award marks.

10. Top Questions to Prepare First (☆ Very Important, across all units)

1. Draw and explain the block diagram of a computer.
2. Explain the memory hierarchy with RAM vs ROM.
3. Differentiate between Compiler and Interpreter.
4. Write an algorithm, pseudocode, and flowchart for a simple problem (e.g., largest of 3 numbers).
5. Differentiate between mutable and immutable data types with examples.
6. Explain operator precedence and associativity with an example.
7. Explain string slicing and any five string methods with examples.
8. Compare List, Tuple, Set, and Dictionary with a table and examples.
9. Write a Python program to implement Stack and Queue using a list.
10. Write a recursive function for factorial and compare recursion with iteration.
11. Write a Python program using a class with a constructor to create multiple objects.
12. Explain decision control and loop control statements with programs.

End of Notes — covers Semester I, Course I: Introduction to Computer Science and Python Programming, all four units and the I-Lab program list, exactly as per the provided syllabus.